

Как да напишем city.cpp

ръководство стъпка по стъпка

Microsoft Visual Studio · C++17 · sort, unique, remove, изключения

Какво ще направим

Ще напишем програма, която чете списък от лица от файл и намира **дублираните** — записи с едно и също ЕГН. После ги премахва по два различни начина.

Темата изглежда като задача за списъци, но всъщност е за едно-единствено решение: **кога два обекта са „един и същ“**. Отговорът се дава в `operator==` и оттам се определя всичко останало — какво намира `find`, какво маха `unique`, какво изтрива `remove`.

Клас	Какво пази
<code>CPerson</code>	Име и ЕГН. ЕГН е уникалният идентификатор .
<code>CCity</code>	Име на града и <code>vector<CPerson></code> .

План на работата

1. Нов проект, скелет и входният файл
2. Класът `CPerson` (изискване I.1)
3. Операторите `<` и `==` — най-важното решение (изисквания I.2–I.3)
4. Операторите `<<` и `>>` (изисквания I.4–I.5)
5. Класът `CCity` и четенето от поток (изисквания II.2, II.6)
6. Конструктор, който хвърля изключение (изискване II.1)
7. Намиране на дубликатите (изискване II.3)
8. Премахване — двата различни начина (изисквания II.4–II.5)
9. Главна функция (изискване III)

1 Нов проект, скелет и входният файл

Нов **Empty Project**, например `Tema10_City`, файл `city.cpp`.

```
#include <algorithm> // sort, unique, remove, find, count
#include <cstdio>     // getchar
#include <fstream>
#include <iomanip>
#include <iostream>
#include <stdexcept> // runtime_error
#include <string>
```

```
#include <vector>
using namespace std;
```

В папката на проекта създайте `Тема10.txt` с данните от условието:

```
Varna
EGN0000001 a
EGN0000001 b
EGN0000002 c
EGN0000003 d
EGN0000004 e
EGN0000004 f
EGN0000005 g
EGN0000006 h
EGN0000001 i
```

Разгледайте данните, преди да пишете код. Първият ред е името на града. После — по един запис на ред, **първо ЕГН, после име**. EGN0000001 се среща три пъти (a, b, i), EGN0000004 — два пъти (e, f). Значи от девет записа **пет** са с повтарящо се ЕГН, а различните ЕГН са шест.

Запомнете тези числа — по тях ще проверявате дали функциите работят.

2 Класът `CPerson`

```
class CPerson
{
private:
    string m_strName;    // име
    string m_strEGN;     // уникален идентификатор (ЕГН)

public:
    CPerson();            // нужен за вектора
    CPerson(const string& strEGN, const string& strName);    // I.1 експлицитен

    const string& getName() const;
    const string& getEGN() const;

    bool operator<(const CPerson& pers) const;    // I.2
    bool operator==(const CPerson& pers) const;    // I.3
    friend ostream& operator<<(ostream& toStream, const CPerson& pers);    // I.4
    friend istream& operator>>(istream& fromStream, CPerson& pers);    // I.5
};
```

Условието иска само експлицитен конструктор, но конструктор по подразбиране пак е нужен. Причини две: `CPerson pers;` във функцията за четене (трябва обект, в който да четем) и вътрешната работа на контейнерите.

Ако напишете само експлицитния, компилаторът **спира** да генерира конструктор по подразбиране и `CPerson pers;` дава `C2512: no appropriate default constructor available`.

Редът на параметрите следва файла: `CPerson(const string& strEGN, const string& strName)` — първо ЕГН. Изкушаващо е да се напише „първо име“, защото така се говори. Но тогава при всяко създаване трябва да се внимава, а данните са в обратния ред. Нека кодът следва данните.

3 Операторите `<` и `==` — най-важното решение

```
// 2. Подредба по ЕГН
bool CPerson::operator<(const CPerson& pers) const
{
    return m_strEGN < pers.m_strEGN;
}

// 3. Равенство САМО по ЕГН
// ЕГН е уникалният идентификатор: двама души с едно ЕГН са едно лице,
// независимо как са изписани имената им.
bool CPerson::operator==(const CPerson& pers) const
{
    return m_strEGN == pers.m_strEGN;
}
```

Тук се решава цялата задача. Изкушаващо е `==` да сравнява *всички* членове — така се пише обикновено. Но условието казва „търсене по ЕГН“ и „дублирани по ЕГН“, а ЕГН е уникален идентификатор. Значи:

```
CPerson("EGN0000001", "a") == CPerson("EGN0000001", "b")    // ИСТИНА
```

Имената са различни, но това е **едно и също лице** — вероятно записано два пъти с различно изписване. Точно тези записи търсим.

Ако `==` сравняваше и името, програмата нямаше да намери нито един дубликат в примерния файл — всичките девет записа са с различни имена.

Следствие, за което трябва да се внимава. Оттук нататък `find`, `count`, `unique` и `remove` ще работят **по ЕГН**, защото всички те използват `operator==`. Това е удобно, но означава, че `remove` ще махне *всички* записи с дадено ЕГН, а не само конкретния обект, който сте подали. В стъпка 8 това е точно каквото искаме — но не приемайте, че винаги е така.

Тук `<` и `==` са **съгласувани**: и двата гледат само ЕГН. Ако нито `a < b`, нито `b < a`, то `a == b`. Това е важно — `unique` в стъпка 8 разчита точно на него.

4 Операторите `<<` и `>>`

```
ostream& operator<<(ostream& toStream, const CPerson& pers)
{
    toStream << setw(12) << left << pers.m_strEGN << " " << pers.m_strName;
    return toStream;
}

// Форматът на файла е <ЕГН> <име> – в този ред.
```

```
istream& operator>>(istream& fromStream, CPerson& pers)
{
    string strEGN, strName;
    if (fromStream >> strEGN >> strName)
    {
        pers.m_strEGN = strEGN;
        pers.m_strName = strName;
    }
    return fromStream;
}
```

Правилата са същите като в упражнения 4 и 5. Тук `setw` работи без помощна функция, защото данните са на латиница.

Проверката `if (fromStream >> ...)` не е излишна. В края на файла четенето се проваля; без проверката ще запишете в обекта каквото е останало в локалните променливи — тоест празни низове — и ще добавите фалшив запис към вектора.

5 Класът `CCity` и четенето от поток

```
class CCity
{
private:
    string          m_strName;          // име на града
    vector<CPerson> m_vecPersons;       // вектор от обекти от I

public:
    CCity();
    explicit CCity(const string& strFileName);    // II.1

    const string& getName() const;
    const vector<CPerson>& getPersons() const;
    size_t size() const;

    ostream& Output(ostream& toStream) const;    // II.2
    vector<CPerson> getDuplicates() const;        // II.3
    void removeDuplicates();                      // II.4
    void removeVector(const vector<CPerson>& toRemove);    // II.5

    friend ostream& operator<<(ostream& toStream, const CCity& city);
    friend istream& operator>>(istream& fromStream, CCity& city);    // II.6
};
```

ЧЕТЕНЕТО НА ЦЯЛ ГРАД

```
istream& operator>>(istream& fromStream, CCity& city)
{
    // Първият ред е името на града, после по един запис на ред.
    if (!(fromStream >> city.m_strName))
        return fromStream;

    city.m_vecPersons.clear();
    CPerson pers;
    while (fromStream >> pers)
```

```

        city.m_vecPersons.push_back(pers);

    return fromStream;
}

```

Три реда вършат цялото четене. `while (fromStream >> pers)` използва оператора на `CPerson` от стъпка 4 — той връща потока, а потокът дава „лъжа“ в края на файла. Броят записи не се знае предварително и не е нужно да се знае: `push_back` расте, доколкото трябва.

`clear()` в началото е предпазна мярка: ако четем два пъти в един обект, старите данни трябва да изчезнат.

ИЗВЕЖДАНЕТО

```

ostream& CCity::Output(ostream& toStream) const
{
    toStream << m_strName << " (" << m_vecPersons.size() << " лица)" << endl;
    for (size_t i = 0; i < m_vecPersons.size(); i++)
        toStream << "    " << m_vecPersons[i] << endl;
    return toStream;
}

ostream& operator<<(ostream& toStream, const CCity& city)
{
    return city.Output(toStream);
}

```

Условието иска и двете — затова операторът просто делегира на `Output`.

6 Конструктор, който хвърля изключение

```

CCity::CCity(const string& strFileName)
    : m_strName("(няма град)", m_vecPersons())
{
    ifstream iFile(strFileName.c_str());
    if (!iFile)
        throw runtime_error("Не мога да отворя файла: " + strFileName);

    iFile >> *this;           // използваме оператора за четене

    if (m_vecPersons.empty())
        throw runtime_error("Файлът " + strFileName + " не съдържа записи");
}

```

ЗАЩО ИЗКЛЮЧЕНИЕ, А НЕ `BOOL`

Конструкторът не може да върне стойност. Ако файлът не се отвори, единствените възможности са: обектът да остане празен и никой да не разбере, или да се хвърли изключение. Условието изрично казва „генерира изключение“.

Ползата е конкретна: обект от `CCity` или е валиден, или изобщо не съществува. Няма трето състояние, за което да се проверява после.

ТУК ПОЛЗВАМЕ `RUNTIME_ERROR`, НЕ СВОЙ КЛАС

В упражнение 6 написахме собствен `CCellException`, защото искахме да видим как се прави. Тук ползваме готовия `std::runtime_error` от `<stdexcept>` — той вече приема низ в конструктора и го връща от `what()`. Собствен клас се пише, когато `catch` трябва да *различава* вашите грешки от чуждите; за едно съобщение готовият е напълно достатъчен.

И двата се улавят от `catch (const exception& ex)`, защото и двата са наследници на `std::exception`.

Деструкторът не се вика при изключение от конструктор. Обектът никога не е бил построен докрай. Вече построените членове (тук `m_strName` и `m_vecPersons`) се унищожават коректно сами. Затова ресурсите се пазят в членове, а не в голи указатели — иначе изключение от конструктор значи сигурен теч.

`ifstream` също се затваря сам при излизане от функцията, включително по изключение.

7 Намиране на дубликатите

```
// Работим върху КОПИЕ: подредбата на член-променливата не се променя.
vector<CPerson> CCity::getDuplicates() const
{
    vector<CPerson> vecSorted = m_vecPersons;
    sort(vecSorted.begin(), vecSorted.end());           // по ЕГН, operator<

    vector<CPerson> vecResult;
    size_t i = 0;
    while (i < vecSorted.size())
    {
        // Намираме докъде се простира групата с едно и също ЕГН.
        size_t j = i + 1;
        while (j < vecSorted.size() && vecSorted[j] == vecSorted[i])
            j++;

        if (j - i > 1)                                // групата е с повече от един
            for (size_t k = i; k < j; k++)
                vecResult.push_back(vecSorted[k]);

        i = j;
    }
    return vecResult;
}
```

ИДЕЯТА: СОРТИРАЙ, ПОСЛЕ ГРУПИРАЙ

След `sort` всички записи с едно ЕГН са **съседни**. Тогава намирането на групи е едно обхождане: `i` сочи началото на групата, вътрешният цикъл придвижва `j` до края ѝ. Ако групата е с повече от един елемент, целият ѝ състав отива в резултата.

Защо не се брои с `count` за всеки елемент? Може — и в главната функция го правим точно така, за да се види. Но `count` обхожда целия вектор за всеки елемент: при n записа това са n^2 сравнения. Сортирането е $n \cdot \log n$, а групирането — още n . За девет записа няма значение; за деветдесет хиляди има.

Функцията е `const` и работи върху копие. Двете вървят заедно: `sort` променя реда, а член-променливата не бива да се пипа — извикващият не очаква, че „да питаш кои са дубликатите“ ще пренареди данните. Затова първият ред прави копие.

Ако забравите копието и сортирате `m_vecPersons` направо, компилаторът ще ви спре: в `const` функция членът е константен.

8 Премахване — двата различни начина

II.4: ОСТАВЯ ПО ЕДИН ОТ ВСЯКО ЕГН

```
void CCity::removeDuplicates()
{
    // unique премахва само СЪСЕДНИ повторения – затова първо sort.
    sort(m_vecPersons.begin(), m_vecPersons.end());

    vector<CPerson>::iterator itNewEnd =
        unique(m_vecPersons.begin(), m_vecPersons.end());

    // unique не смалява вектора – връща новия край. Изтриваме опашката.
    m_vecPersons.erase(itNewEnd, m_vecPersons.end());
}
```

unique не маха нищо. Това е най-подвеждащото име в цялата стандартна библиотека. Алгоритъмът **размества**: избутва оставащите елементи в началото и връща итератор към първия ненужен. Векторът остава със същия размер, а в опашката стои боклук.

преди unique:	1 1 1 2 3 4 4 5 6	size() == 9
след unique:	1 2 3 4 5 6 ? ? ?	size() == 9, itNewEnd сочи тук ^
след erase:	1 2 3 4 5 6	size() == 6

Без реда с `erase` векторът ще изглежда непроменен по размер, а в края ще има повторени стойности. Двата реда заедно се наричат **идиом erase-unique** и се пишат винаги заедно.

sort преди unique е задължителен. `unique` сравнява всеки елемент само със *съседния*. Върху несортирани данни `1 2 1` няма да намери нищо. Сортирането ги прави съседни.

II.5: МАХА ПОДАДЕНИТЕ ЗАПИСИ

```
// operator== сравнява по ЕГН, значи се маха ВСЯКО лице с това ЕГН.
void CCity::removeVector(const vector<CPerson>& toRemove)
{
    for (size_t i = 0; i < toRemove.size(); i++)
    {
        // Идиомът remove-erase: remove избутва ненужните в края
        // и връща новия край; erase ги изтрива наистина.
        m_vecPersons.erase(
            remove(m_vecPersons.begin(), m_vecPersons.end(), toRemove[i]),
            m_vecPersons.end());
    }
}
```

```
}  
}
```

remove също не премахва нищо — по същата причина като `unique`. Двойката `erase(remove(...), end())` е **идиомът remove-erase**, вторият от двата, които трябва да се запомнят наизуст.

Причината и двата да работят така: алгоритмите получават само итератори и нямат достъп до контейнера. Те не знаят как да го смалят — само как да разместят елементи. Смаляването е работа на контейнера, тоест на `erase`.

ЗАЩО ДВЕТЕ ДАВАТ РАЗЛИЧЕН РЕЗУЛТАТ

	<code>removeDuplicates()</code>	<code>removeVector</code> (дубликатите)
Какво прави	оставя по един от всяко ЕГН	маха всички записи с дублирано ЕГН
Остават	6 лица	4 лица
EGN0000001 (3 запис)	остава един	изчезва напълно
EGN0000002 (1 запис)	остава	остава

Второто е по-грубо: то не „изчиства“, а **изхвърля** всичко съмнително. И двете са смислени — зависи какво се иска.

Забележете защо `removeVector` маха и трите запис с **EGN0000001**, въпреки че подаваме три отделни обекта. Първото извикване на `remove` маха *всички* с това ЕГН наведнъж, защото `operator==` гледа само ЕГН. Следващите две извиквания просто не намират нищо. Резултатът е верен, но пътят до него е по-кратък, отколкото изглежда.

9 Главна функция

9.1 Двата случая при създаване от файл

```
// а) несъществуващ файл  
try  
{  
    CCity cityBad("няма-такъв-файл.txt");  
    cout << " създаден: " << cityBad.getName() << endl;  
}  
catch (const exception& ex)  
{  
    cout << " хванато изключение: " << ex.what() << endl;  
}  
  
// б) истинският файл  
CCity city;  
try  
{  
    city = CCity("Тема10.txt");  
    cout << " прочетен град: " << city.getName()  
        << ", лица: " << city.size() << endl;  
}
```

```

    }
    catch (const exception& ex)
    {
        cout << " хванато изключение: " << ex.what() << endl;
        cout << " Без данни програмата няма какво да прави." << endl;
        getchar();
        return 1;
    }

```

Двата `catch` реагират различно, и това е смисълът. В случай а) очакваме грешката — съобщаваме и продължаваме. В случай б) без данни няма какво да се прави, затова програмата приключва с код 1 . Изключението не предписва какво да стане; то само съобщава, а решението е на този, който го улавя.

`city` се обявява **извън** `try` , за да е видим и след него — обект, създаден вътре в блока, изчезва на затварящата скоба.

9.2 Броене с `count`

```
long nCount = count(vecAll.begin(), vecAll.end(), vecAll[i]);
```

`count` преброява елементите, равни на дадения — пак чрез `operator==` , значи по ЕГН. Използваме го, за да покажем по колко пъти се среща всяко ЕГН; списъкът `vecSeen` пази вече отчетените, за да не се повтарят.

9.3 Работа върху копия

```

CCity cityA = city;                                // работим върху копие
cityA.removeDuplicates();

CCity cityB = city;                                // пак от оригинала
cityB.removeVector(vecDup);

```

Копирането е безплатно за нас. `CCity` няма нито деструктор, нито копиращ конструктор, нито `operator=` — генерираните са коректни, защото `string` и `vector` се копират правилно сами. Сравнете с упражнение 3, където същото изискваше три написани на ръка функции.

Така двете операции се прилагат върху едни и същи начални данни и резултатите са сравними.

9.4 Търсене по ЕГН

```

CPerson target(arrSearch[i], "");    // името няма значение за ==
vector<CPerson>::const_iterator it = find(v.begin(), v.end(), target);

if (it != v.end())
    cout << "намерен: " << *it << " (позиция " << (it - v.begin()) << ")" << endl;
else
    cout << "НЕ е намерен" << endl;

```

Търсим с обект, чието име е празно — и пак намираме. Това е пряко следствие от `operator==` : името не участва в сравнението. Обектът `target` служи само като носител на ЕГН-то, което търсим.

`it - v.begin()` дава позицията като число. Тази аритметика работи за `vector` , защото елементите му лежат последователно в паметта. За `list` не работи — там се ползва `distance(v.begin(), it)` .

Компилиране и стартиране

Клавиши	Действие
<code>Ctrl</code> + <code>Shift</code> + <code>B</code>	Build.
<code>F5</code>	Start Debugging. Работната папка е тази на проекта — оттам се търси <code>Тема10.txt</code> .
<code>F9</code>	Точка на прекъсване след <code>unique</code> . В Watch вижте, че <code>m_vecPersons.size()</code> още е 9, а в края стои боклук — това е най-ясната илюстрация на стъпка 8.

```
cl /EHsc /std:c++17 /utf-8 city.cpp
city.exe
```

Кирилица под Windows

Съобщенията в програмата са на кирилица, макар данните да са на латиница. Файлът се записва като *Unicode (UTF-8 with signature)* — *Codepage 65001*; компилаторът получава `/utf-8` ; конзолата се превключва с `chcp 65001` или със `system("chcp 65001 > nul");` като първи ред в `main` .

Тема10.txt остава на латиница. Данните идват от условието в този вид и така се четат еднакво независимо от кодировката на файла. Ако сложите кирилица в него, запишете го в UTF-8 — и имайте предвид, че `setw(12)` в `operator<<` ще брой байтове (виж упражнение 6).

Пълен код на програмата

Ако някъде сте се загубили, сравнете с този листинг:

```
// Тема 10: STL, контейнери, итератори, алгоритми.
// CPerson – личност с име и ЕГН.
// CCity – град с вектор от личности; търсене и премахване на дубликати.

#include <algorithm> // sort, unique, remove, find, count
#include <cstdio>     // getchar
#include <fstream>
#include <iomanip>
#include <iostream>
#include <stdexcept> // runtime_error
#include <string>
#include <vector>
using namespace std;

// =====
// I. Клас CPerson
// =====
class CPerson
{
// Скрита член променлива
private:
    string m_strName;    // име
    string m_strEGN;     // уникален идентификатор (ЕГН)

public:
    CPerson();           // нужен за вектора
    // 1. Експлицитен конструктор
    CPerson(const string& strEGN, const string& strName);

    // Функции за достъп – само тези, които наистина трябва
    const string& getName() const;
    const string& getEGN() const;

    // 2. Сортиране по ЕГН в нарастващ ред
    bool operator<(const CPerson& pers) const;

    // 3. Търсене по ЕГН
    bool operator==(const CPerson& pers) const;

    // 4. Извеждане в поток
    friend ostream& operator<<(ostream& toStream, const CPerson& pers);

    // 5. Четене от поток
    friend istream& operator>>(istream& fromStream, CPerson& pers);
};

CPerson::CPerson() : m_strName("(няма)"), m_strEGN("") {}

CPerson::CPerson(const string& strEGN, const string& strName)
    : m_strName(strName), m_strEGN(strEGN) {}

const string& CPerson::getName() const { return m_strName; }
```

```

const string& CPerson::getEGN() const { return m_strEGN; }

// ----- 2. Подредба по ЕГН -----
bool CPerson::operator<(const CPerson& pers) const
{
    return m_strEGN < pers.m_strEGN;
}

// ----- 3. Равенство САМО по ЕГН -----
// ЕГН е уникалният идентификатор: двама души с едно ЕГН са едно лице,
// независимо как са изписани имената им.
bool CPerson::operator==(const CPerson& pers) const
{
    return m_strEGN == pers.m_strEGN;
}

// ----- 4. Извеждане -----
ostream& operator<<(ostream& toStream, const CPerson& pers)
{
    toStream << setw(12) << left << pers.m_strEGN << " " << pers.m_strName;
    return toStream;
}

// ----- 5. Четене -----
// Форматът на файла е <ЕГН> <име> – в този ред.
istream& operator>>(istream& fromStream, CPerson& pers)
{
    string strEGN, strName;
    if (fromStream >> strEGN >> strName)
    {
        pers.m_strEGN = strEGN;
        pers.m_strName = strName;
    }
    return fromStream;
}

// =====
// II. Клас CCity
// =====
class CCity
{
private:
    string          m_strName;          // име на града
    vector<CPerson> m_vecPersons;       // вектор от обекти от I

public:
    CCity();
    // 1. Експлицитен конструктор с параметър име на файл; хвърля изключение
    explicit CCity(const string& strFileName);

    const string& getName() const;
    const vector<CPerson>& getPersons() const;
    size_t size() const;

    // 2. Извеждане на вектора на изходен поток
    ostream& Output(ostream& toStream) const;

    // 3. Връща дублираните по ЕГН лица
    vector<CPerson> getDuplicates() const;

```

```

// 4. Премахва дубликатите от член-променливата (оставя по един)
void removeDuplicates();

// 5. Премахва от вектора елементите на подадения вектор
void removeVector(const vector<CPerson>& toRemove);

friend ostream& operator<<(ostream& toStream, const CCity& city);
// 6. Четене от поток
friend istream& operator>>(istream& fromStream, CCity& city);
};

CCity::CCity() : m_strName("(няма град)'), m_vecPersons() {}

// ----- 1. Конструктор по име на файл -----
CCity::CCity(const string& strFileName)
    : m_strName("(няма град)'), m_vecPersons()
{
    ifstream iFile(strFileName.c_str());
    if (!iFile)
        throw runtime_error("Не мога да отворя файла: " + strFileName);

    iFile >> *this;          // използваме оператора за четене

    if (m_vecPersons.empty())
        throw runtime_error("Файлът " + strFileName + " не съдържа записи");
}

const string& CCity::getName() const           { return m_strName; }
const vector<CPerson>& CCity::getPersons() const { return m_vecPersons; }
size_t CCity::size() const                     { return m_vecPersons.size(); }

// ----- 2. Извеждане -----
ostream& CCity::Output(ostream& toStream) const
{
    toStream << m_strName << " (" << m_vecPersons.size() << " лица)" << endl;
    for (size_t i = 0; i < m_vecPersons.size(); i++)
        toStream << "    " << m_vecPersons[i] << endl;
    return toStream;
}

ostream& operator<<(ostream& toStream, const CCity& city)
{
    return city.Output(toStream);
}

// ----- 6. Четене -----
istream& operator>>(istream& fromStream, CCity& city)
{
    // Първият ред е името на града, после по един запис на ред.
    if (!(fromStream >> city.m_strName))
        return fromStream;

    city.m_vecPersons.clear();
    CPerson pers;
    while (fromStream >> pers)
        city.m_vecPersons.push_back(pers);

    return fromStream;
}

```

```

}

// ----- 3. Дублираните по ЕГН лица -----
// Работим върху КОПИЕ: подредбата на член-променливата не се променя.
vector<CPerson> CCity::getDuplicates() const
{
    vector<CPerson> vecSorted = m_vecPersons;
    sort(vecSorted.begin(), vecSorted.end());          // по ЕГН, operator<

    vector<CPerson> vecResult;
    size_t i = 0;
    while (i < vecSorted.size())
    {
        // Намираме докъде се простира групата с едно и също ЕГН.
        size_t j = i + 1;
        while (j < vecSorted.size() && vecSorted[j] == vecSorted[i])
            j++;

        if (j - i > 1)                                // групата е с повече от един
            for (size_t k = i; k < j; k++)
                vecResult.push_back(vecSorted[k]);

        i = j;
    }
    return vecResult;
}

// ----- 4. Премахване на дубликатите -----
// Остава по ЕДИН запис за всяко ЕГН.
void CCity::removeDuplicates()
{
    // unique премахва само СъСЕДНИ повторения – затова първо sort.
    sort(m_vecPersons.begin(), m_vecPersons.end());

    vector<CPerson>::iterator itNewEnd =
        unique(m_vecPersons.begin(), m_vecPersons.end());

    // unique не смалява вектора – връща новия край. Изтриваме опашката.
    m_vecPersons.erase(itNewEnd, m_vecPersons.end());
}

// ----- 5. Премахване на подадените елементи -----
// operator== сравнява по ЕГН, значи се маха ВСЯКО лице с това ЕГН.
void CCity::removeVector(const vector<CPerson>& toRemove)
{
    for (size_t i = 0; i < toRemove.size(); i++)
    {
        // Идиомът remove-erase: remove избутва ненужните в края
        // и връща новия край; erase ги изтрива наистина.
        m_vecPersons.erase(
            remove(m_vecPersons.begin(), m_vecPersons.end(), toRemove[i]),
            m_vecPersons.end());
    }
}

// =====
// III. Главна функция
// =====
int main()

```

```

{
// ===== 1. Създаване от файл с обработка на изключение =====
cout << "===== 1. Създаване на обект от файл =====" << endl;

// а) несъществуващ файл
try
{
    CCity cityBad("няма-такъв-файл.txt");
    cout << "    създаден: " << cityBad.getName() << endl;
}
catch (const exception& ex)
{
    cout << "    хванато изключение: " << ex.what() << endl;
}

// б) истинският файл
CCity city;
try
{
    city = CCity("Тема10.txt");
    cout << "    прочетен град: " << city.getName()
        << ", лица: " << city.size() << endl;
}
catch (const exception& ex)
{
    cout << "    хванато изключение: " << ex.what() << endl;
    cout << "    Без данни програмата няма какво да прави." << endl;
    getchar();
    return 1;
}

// ===== 2. Извеждане =====
cout << endl << "===== 2. Извеждане (operator<<) =====" << endl;
cout << city;

// ===== 3. Дублираните по ЕГН лица =====
cout << endl << "===== 3. Дублирани по ЕГН =====" << endl;
vector<CPerson> vecDup = city.getDuplicates();
cout << "    намерени " << vecDup.size() << " записа с повтарящо се ЕГН:" << endl;
for (size_t i = 0; i < vecDup.size(); i++)
    cout << "        " << vecDup[i] << endl;

// Колко пъти се среща всяко ЕГН – с алгоритъма count
cout << "---- Брой повторения (count) ----" << endl;
const vector<CPerson>& vecAll = city.getPersons();
vector<string> vecSeen;
for (size_t i = 0; i < vecAll.size(); i++)
{
    if (find(vecSeen.begin(), vecSeen.end(), vecAll[i].getEGN()) != vecSeen.end())
        continue; // вече сме броили това ЕГН
    vecSeen.push_back(vecAll[i].getEGN());

    long nCount = count(vecAll.begin(), vecAll.end(), vecAll[i]);
    cout << "        " << setw(12) << left << vecAll[i].getEGN()
        << " -> " << nCount << (nCount > 1 ? "    <- дубликат" : "") << endl;
}

// ===== 4. Премахване на дубликатите (остава по един) =====
cout << endl << "===== 4. removeDuplicates() =====" << endl;

```

```

CCity cityA = city; // работим върху копие
cout << " преди: " << cityA.size() << " лица" << endl;
cityA.removeDuplicates();
cout << " след: " << cityA.size() << " лица" << endl;
cout << cityA;

// ===== 5. Премахване на всички дублирани лица =====
cout << endl << "===== 5. removeVector(дубликатите) =====" << endl;
CCity cityB = city; // пак от оригинала
cout << " преди: " << cityB.size() << " лица" << endl;
cityB.removeVector(vecDup);
cout << " след: " << cityB.size() << " лица" << endl;
cout << cityB;

// ===== Разликата между 4 и 5 =====
cout << endl << "===== Разликата между 4 и 5 =====" << endl;
cout << " оригинал: " << city.size() << " лица" << endl;
cout << " removeDuplicates: " << cityA.size() << " лица (оставя по един от всяко ЕГН)" <<
endl;
cout << " removeVector: " << cityB.size() << " лица (маха всички дублирани изцяло)"
<< endl;

// ===== Търсене по ЕГН =====
cout << endl << "===== Търсене по ЕГН (find + operator==) =====" << endl;
const char* arrSearch[] = { "EGN0000003", "EGN0000001", "EGN9999999" };
for (size_t i = 0; i < sizeof(arrSearch) / sizeof(arrSearch[0]); i++)
{
    CPerson target(arrSearch[i], ""); // името няма значение за ==
    const vector<CPerson>& v = city.getPersons();
    vector<CPerson>::const_iterator it = find(v.begin(), v.end(), target);

    cout << " " << setw(12) << left << arrSearch[i] << " -> ";
    if (it != v.end())
        cout << "намерен: " << *it << " (позиция " << (it - v.begin()) << ")" << endl;
    else
        cout << "НЕ е намерен" << endl;
}

// ===== Сортиране на оригинала =====
cout << endl << "===== Сортиране по ЕГН (operator<) =====" << endl;
CCity cityC = city;
vector<CPerson> vecSorted = cityC.getPersons();
sort(vecSorted.begin(), vecSorted.end());
for (size_t i = 0; i < vecSorted.size(); i++)
    cout << " " << vecSorted[i] << endl;

// Задържа конзолния прозорец отворен при стартиране с F5 (debug)
getchar();
return 0;
}

```

Очакван изход

```

===== 1. Създаване на обект от файл =====
хванато изключение: Не мога да отворя файла: няма-такъв-файл.txt
прочетен град: Varna, лица: 9

```

===== 2. Извеждане (operator<<) =====

Varna (9 лица)

EGN0000001	a
EGN0000001	b
EGN0000002	c
EGN0000003	d
EGN0000004	e
EGN0000004	f
EGN0000005	g
EGN0000006	h
EGN0000001	i

===== 3. Дублирани по ЕГН =====

намерени 5 записа с повтарящо се ЕГН:

EGN0000001	a
EGN0000001	b
EGN0000001	i
EGN0000004	e
EGN0000004	f

--- Брой повторения (count) ---

EGN0000001	-> 3	<- дубликат
EGN0000002	-> 1	
EGN0000003	-> 1	
EGN0000004	-> 2	<- дубликат
EGN0000005	-> 1	
EGN0000006	-> 1	

===== 4. removeDuplicates() =====

преди: 9 лица

след: 6 лица

Varna (6 лица)

EGN0000001	a
EGN0000002	c
EGN0000003	d
EGN0000004	e
EGN0000005	g
EGN0000006	h

===== 5. removeVector(дубликатите) =====

преди: 9 лица

след: 4 лица

Varna (4 лица)

EGN0000002	c
EGN0000003	d
EGN0000005	g
EGN0000006	h

===== Разликата между 4 и 5 =====

оригинал: 9 лица

removeDuplicates: 6 лица (оставя по един от всяко ЕГН)

removeVector: 4 лица (маха всички дублирани изцяло)

===== Търсене по ЕГН (find + operator==) =====

EGN0000003	-> намерен: EGN0000003	d	(позиция 3)
EGN0000001	-> намерен: EGN0000001	a	(позиция 0)
EGN9999999	-> НЕ е намерен		

===== Сортиране по ЕГН (operator<) =====

EGN0000001	a
EGN0000001	b
EGN0000001	i
EGN0000002	c
EGN0000003	d
EGN0000004	e
EGN0000004	f
EGN0000005	g
EGN0000006	h

Чести грешки

Код	Съобщение	Причина и решение
C2512	'CPerson': no appropriate default constructor available	Написан е само експлицитният конструктор. Добавете и подразбиращ се (стъпка 2).
C2678	binary '==': no operator found which takes a left-hand operand of type 'const CPerson'	<code>operator==</code> не е маркиран с <code>const</code> . Алгоритмите работят с константни елементи.
C3892	you cannot assign to a variable that is const	<code>sort</code> върху член-променлива в <code>const</code> функция. Сортирайте копие (стъпка 7).
C2664	cannot convert argument 1 from 'std::string' to ...	<code>ifstream</code> при по-стари компилатори иска <code>const char*</code> . Оттам е <code>.c_str()</code> .
—	<code>size()</code> не се променя след <code>unique</code>	Липсва <code>erase</code> . Двата реда се пишат заедно (стъпка 8).
—	В края на вектора стоят повторени записи	Същото — липсва <code>erase</code> .
—	<code>unique</code> не намира дубликати	Липсва <code>sort</code> преди него. <code>unique</code> гледа само съседни елементи.
—	Намерени са 0 дубликата	<code>operator==</code> сравнява и името. Трябва да сравнява само ЕГН (стъпка 3).
—	Прочетени са 0 лица	<code>Tema10.txt</code> не е в работната папка, или редът на полетата е обърнат (първо ЕГН, после име).
—	Последният запис е празен	Липсва проверката <code>if (fromStream >> ...)</code> в <code>operator>></code> (стъпка 4).
—	Програмата спира с <i>unhandled exception</i>	Създаването на <code>CCity</code> не е в <code>try</code> блок.

Обобщение: четирите алгоритъма

Алгоритъм	Какво прави	Ползва	Капан
<code>sort</code>	подрежда	<code>operator<</code>	работи за <code>vector</code> , не за <code>list</code>
<code>find</code>	връща итератор към първото съвпадение или <code>end()</code>	<code>operator==</code>	връщаното <code>end()</code> винаги се проверява

`unique` избутва повторенията в края

`operator==` иска сортирано; не смаява —
нужен е `erase`

`remove` избутва съпадащите в края

`operator==` не смаява — нужен е `erase`

Общото между четирите: нито един не знае нищо за `CPerson`. Те работят с итератори и с двата оператора, които вие сте написали. Каквото сте сложили в `operator==`, това ще търсят и това ще махат.

Какво следва

Упражнение 11 `list` — контейнер без произволен достъп, с два успоредни списъка и собствен `sort`.

Упражнение 12 `map` и `multimap` — вместо да търсите с `find` по целия контейнер, ключът ви отвежда направо.

Упражнение 13 `set` — контейнер, който **сам** не допуска дубликати. Цялата стъпка 8 отпада.